BLACK PEN Answer all questions first. GREEN PEN Then: tick correct answers; rewrite mistakes.

LESSON 045 · Data Structures · Constructors and Encapsulation

Name: _____ Class: _____ Date: _____

Time: _____ Teacher: _____

Work through the pages in order. Answer in BLACK PEN first. Check in GREEN PEN only after the Exit Ticket.

Do Now · Recall previous learning

Q1. How do you create an object in C#?

Q2. What is an instance field?

Q3. What does modularity improve?

Q4. What is LIFO?

Do Now Recall previous learning | **I Do** Read new knowledge | **We Do** Practise with guidance | **You Do** Work independently | **Review & Improve** Check my understanding

BLACK PEN Answer all questions first. GREEN PEN Then: tick correct answers; rewrite mistakes.

I Do · Read new knowledge

Key vocabulary

Recall the term — constructor

Recall the term — encapsulation

Recall the term — private

Recall the term — public

Recall the term — validation

New learning

Protect the object invariant — The constructor refuses a negative opening balance before storing it. The field is private, so outside code must use approved public methods instead of bypassing validation.

Worked example

Construct a valid account; control updates

1. Constructor sets the initial state — new Account(10) invokes Account(int opening) . It checks 10 is non-negative, then stores 10 in the private field balance .

2. Method validates the proposed change — savings.Deposit(5) passes the positive-amount check and increases balance from 10 to 15. Outside code cannot change the private field directly.

3. Check valid and invalid cases — GetBalance() returns 15. Deposit(-3) throws an exception before changing the balance. Encapsulation is controlled access, not merely writing private .

We Do · Practise with guidance

Trace a different validated account

Using the Account class, run Account current = new Account(20); current.Deposit(7); Then attempt current.Deposit(-2); . What does current.GetBalance() give afterwards if the exception is caught by the caller?

Show the value after each operation

What happens at construction, after the valid deposit, and at the attempted negative deposit?

27; invalid deposit rejected

Constructor sets 20; valid deposit makes 27. The second call raises an ArgumentOutOfRangeException before the assignment; when the caller handles it,

the balance remains 27 .

Construct a different valid object

A Box has private int width; and a constructor accepting width. Write the validation condition that rejects zero or negative widths, then state which values new Box(4) and new Box(0) should allow.

Apply the same pattern

Identify the precondition before any assignment to width .

Reject invalid initial state

if (w <= 0) throw new ArgumentOutOfRangeException(nameof(w)); width = w; goes inside public Box(int w) . Width 4 is accepted; width 0 is rejected.

BLACK PEN Answer all questions first. GREEN PEN Then: tick correct answers; rewrite mistakes.

You Do · Work independently

BLACK PEN: Write your own answers. Do not look at the Review & Improve pages yet.

Question 1

Apply independently
Write a constructor for class Box that stores integer width. [4]

Question 2

Apply independently
Explain encapsulation. [2]

Question 3

Apply independently
Suggest validation for a bank deposit method. [2]

Exit Ticket · Answer before checking

True or false: a constructor has a declared return type void.

BLACK PEN Answer all questions first. GREEN PEN Then: tick correct answers; rewrite mistakes.

Review & Improve · GREEN PEN

Now change to GREEN PEN. Find each original question. Tick correct answers; read the model and rewrite mistakes.

Page 1 onwards · Do Now

Q1. Use new with a constructor

Q2. Value held separately per object

Q3. Reuse and maintainability

Q4. Last in first out

You Do pages · Independent questions

Q1. public Box(int w) { width = w; } (with width declared as an instance field).

Q2. Keeping state inside an object and controlling access through methods/properties.

Q3. Reject amounts <= 0 before adding to balance.

Exit Ticket · Model answer

False. In C#, a constructor is a special member used to initialise a new object. It has the class name and no declared return type—not even void.

Try the question again

Choose one question you got wrong. All correct? Choose the hardest.

1. Copy the whole original question:

2. Write a complete new answer in GREEN PEN. Show working if needed: